

INTEGRATION GUIDE: Tiered Notification System into Existing Code

Step-by-Step Modifications

📍 LOCATION 1: adminpanel/views.py - admin_approve_blood_request()

CURRENT CODE (lines ~120-180):

```
python

@api_view(["POST"])
@permission_classes([AllowAny])
def admin_approve_blood_request(request, request_id):
    try:
        blood_request = BloodRequest.objects.get(id=request_id)
    except BloodRequest.DoesNotExist:
        return Response(
            {"success": False, "message": "Blood request not found"},
            status=404
        )

    if blood_request.status.lower() != "pending":
        return Response(
            {"success": False, "message": "Request already processed"},
            status=400
        )

    blood_request.status = "approved"
    blood_request.patient_confirmed = False
    blood_request.fulfilled = False
    blood_request.save()

    # ... rest of function ...
```

WHAT TO DO:

Replace the entire function body AFTER the status change with this:

```
python
```

```

blood_request.status = "approved"
blood_request.patient_confirmed = False
blood_request.fulfilled = False
blood_request.save()

# ✅ NOTIFY PATIENT
if blood_request.patient:
    patient_user = User.objects.filter(
        username=blood_request.patient.emailaddress
    ).first()
    if patient_user:
        Notification.objects.create(
            user=patient_user,
            blood_request=blood_request,
            title="Blood Request Approved by Admin",
            message=(
                f"Your blood request for {blood_request.blood_type} blood at "
                f"{blood_request.hospital_location.name if blood_request.hospital_location else 'Hospital'} "
                f"has been approved by admin. Searching for donors..."
            ),
            type="blood_request_approved_by_admin"
        )

# 🔴 START TIERED NOTIFICATION SYSTEM HERE
logger.info(f"🚀 Starting tiered notification for request #{blood_request.id}")

# Option A: Using Celery (RECOMMENDED for production)
from celery_tasks import orchestrate_tiered_notification
orchestrate_tiered_notification.delay(blood_request.id)

# Option B: Synchronous (for testing without Celery)
# from celery_tasks import orchestrate_tiered_notification
# result = orchestrate_tiered_notification(blood_request.id)

return Response({
    "success": True,
    "message": "Blood request approved. Tiered notifications started."
})

```

Why this change?

- Removes old single-tier logic

- Triggers Celery task for async tiered notifications
 - Maintains patient notification
 - Enables background processing
-

📌 **LOCATION 2: adminpanel/models.py - ADD NEW MODELS**

ADD THESE TWO NEW MODELS:

```
python
```

```

from django.db import models
from blood_requests.models import BloodRequest
from register_donor.models import Donor
from hospital.models import Hospital

class NotificationLog(models.Model):
    """Track all notifications sent for auditing"""

    TIER_CHOICES = [
        ('tier_1', 'Tier 1 (0-5km)'),
        ('tier_2', 'Tier 2 (5-15km)'),
        ('tier_3', 'Tier 3 (15-30km)'),
        ('tier_4', 'Tier 4 (30km+)'),
    ]

    blood_request = models.ForeignKey(BloodRequest, on_delete=models.CASCADE, related_name='notification_log')
    donor = models.ForeignKey(Donor, on_delete=models.CASCADE, null=True, blank=True)
    tier = models.CharField(max_length=10, choices=TIER_CHOICES)
    distance_km = models.FloatField()
    notification_type = models.CharField(max_length=20, choices=[
        ('sms', 'SMS'),
        ('email', 'Email'),
    ])
    status = models.CharField(max_length=20, choices=[
        ('sent', 'Sent'),
        ('failed', 'Failed'),
    ], default='sent')
    sent_at = models.DateTimeField(auto_now_add=True)
    error_message = models.TextField(blank=True)

    class Meta:
        ordering = ['-sent_at']

class BloodRequestEscalation(models.Model):
    """Track escalation timeline"""

    blood_request = models.OneToOneField(BloodRequest, on_delete=models.CASCADE)
    hospital = models.ForeignKey(Hospital, on_delete=models.SET_NULL, null=True, blank=True)

    tier_1_started = models.DateTimeField(null=True, blank=True)
    tier_1_completed = models.DateTimeField(null=True, blank=True)
    tier_1_donor_count = models.IntegerField(default=0)

```

```
tier_2_started = models.DateTimeField(null=True, blank=True)
tier_2_completed = models.DateTimeField(null=True, blank=True)
tier_2_donor_count = models.IntegerField(default=0)

tier_3_started = models.DateTimeField(null=True, blank=True)
tier_3_completed = models.DateTimeField(null=True, blank=True)
tier_3_donor_count = models.IntegerField(default=0)

tier_4_started = models.DateTimeField(null=True, blank=True)
tier_4_completed = models.DateTimeField(null=True, blank=True)
tier_4_donor_count = models.IntegerField(default=0)

blood_bank_checked = models.DateTimeField(null=True, blank=True)
blood_bank_stock_found = models.BooleanField(default=False)
blood_bank_units = models.IntegerField(default=0)

hospital_stock_checked = models.DateTimeField(null=True, blank=True)
hospital_stock_found = models.BooleanField(default=False)
hospital_stock_details = models.JSONField(default=dict, blank=True)

completed_at = models.DateTimeField(null=True, blank=True)
total_donors_alerted = models.IntegerField(default=0)
success = models.BooleanField(default=False)
```

THEN RUN:

```
bash

python manage.py makemigrations
python manage.py migrate
```

📍 LOCATION 3: blood_requests/views.py - UPDATE is_donor_eligible()

CURRENT CODE (around line 850+):

```
python
```

```
def is_donor_eligible(donor):
    last_donation = (
        Donation.objects
        .filter(donor=donor, status="verified")
        .order_by("-date")
        .first()
    )

    if not last_donation:
        return True

    next_allowed = last_donation.date + timedelta(days=MIN_GAP_DAYS)
    return timezone.now().date() >= next_allowed
```

WHAT TO DO:

ADD THIS FUNCTION RIGHT AFTER is_donor_eligible():

python

```

def is_donor_eligible(donor):
    """Check if donor is eligible to donate (keeps existing logic)"""
    last_donation = (
        Donation.objects
        .filter(donor=donor, status="verified")
        .order_by("-date")
        .first()
    )

    if not last_donation:
        return True

    next_allowed = last_donation.date + timedelta(days=MIN_GAP_DAYS)
    return timezone.now().date() >= next_allowed

# ✅ ADD THIS NEW FUNCTION
def get_compatible_donors(blood_type):
    """Get all blood types that can donate to this blood type"""
    BLOOD_COMPATIBILITY = {
        "O-": ["O-", "O+", "A-", "A+", "B-", "B+", "AB-", "AB+"],
        "O+": ["O+", "A+", "B+", "AB+"],
        "A-": ["A-", "A+", "AB-", "AB+"],
        "A+": ["A+", "AB+"],
        "B-": ["B-", "B+", "AB-", "AB+"],
        "B+": ["B+", "AB+"],
        "AB-": ["AB-", "AB+"],
        "AB+": ["AB+"],
    }
    return [
        donor_blood
        for donor_blood, receivers in BLOOD_COMPATIBILITY.items()
        if blood_type in receivers
    ]

```

Why?

- Reuses your existing eligibility logic
- Adds compatibility mapper needed by tiered system
- Keeps everything in one place

CREATE: `your_project/celery.py`

```
python

import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'your_project.settings')

app = Celery('your_project')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

@app.task(bind=True)
def debug_task(self):
    print(f'Request: {self.request!r}')
```

Why?

- Initializes Celery for async task processing
 - Enables `celery -A your_project worker` command
-

📌 LOCATION 5: Update project/init.py

CURRENT CODE:

```
python

# Usually empty or has nothing
```

ADD THIS:

```
python

from .celery import app as celery_app

__all__ = ('celery_app',)
```

Why?

- Makes Celery app available at project startup
-

📌 LOCATION 6: Create NEW FILE: celery_tasks.py in project root

CREATE: `your_project/celery_tasks.py`

Copy the entire content from the file: `celery_tiered_tasks.py` provided in the implementation package.

This file contains all 6 Celery tasks:

- `orchestrate_tiered_notification`
 - `notify_tier_1`
 - `notify_tier_2`
 - `notify_tier_3`
 - `notify_tier_4`
 - `check_blood_stock`
-

📌 LOCATION 7: Update settings.py - ADD CELERY CONFIG

FIND: Your existing settings.py

ADD THIS SECTION (before the closing of file):

```
python
```

```
# ===== CELERY CONFIGURATION =====
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = 'Asia/Kathmandu'
CELERY_ENABLE_UTC = True

# Task time limits
CELERY_TASK_TIME_LIMIT = 30 * 60 # 30 minutes
CELERY_TASK_SOFT_TIME_LIMIT = 25 * 60 # 25 minutes

# For development/testing (run tasks synchronously)
# CELERY_ALWAYS_EAGER = True
# CELERY_EAGER_PROPAGATES_EXCEPTIONS = True

# ===== BLOOD BANK CONFIG =====
BLOOD_BANK_CONTACT = "+977-1-4428888"
BLOOD_BANK_EMAIL = "bloodbank@hospital.com"

# ===== LOGGING =====
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
        },
    },
    'loggers': {
        'adminpanel': {
            'handlers': ['console'],
            'level': 'INFO',
        },
        'celery': {
            'handlers': ['console'],
            'level': 'INFO',
        },
    },
}
```

📌 LOCATION 8: Update admin_approve_blood_request() - ADD IMPORTS

AT THE TOP of adminpanel/views.py:

```
python

# Add these imports
import logging
from celery_tasks import orchestrate_tiered_notification

logger = logging.getLogger(__name__)
```

📌 LOCATION 9: Optional - ADD API ENDPOINTS

In adminpanel/urls.py or main urls.py:

```
python

from adminpanel.views import admin_escalation_status, admin_notification_logs

urlpatterns = [
    # ... existing urls ...

    # Tiered notification tracking endpoints
    path('api/escalation/<int:request_id>/status/', admin_escalation_status, name='e'),
    path('api/escalation/<int:request_id>/logs/', admin_notification_logs, name='not'),
]
```

ADD THESE FUNCTIONS to adminpanel/views.py:

```
python
```

```

@api_view(["GET"])
@permission_classes([AllowAny])
def admin_escalation_status(request, request_id):
    """Get current escalation status"""
    try:
        escalation = BloodRequestEscalation.objects.get(blood_request_id=request_id)

        return Response({
            "request_id": request_id,
            "status": "escalating" if not escalation.completed_at else "completed",
            "tier_1": {
                "completed": escalation.tier_1_completed is not None,
                "donors_notified": escalation.tier_1_donor_count
            },
            "tier_2": {
                "completed": escalation.tier_2_completed is not None,
                "donors_notified": escalation.tier_2_donor_count
            },
            "tier_3": {
                "completed": escalation.tier_3_completed is not None,
                "donors_notified": escalation.tier_3_donor_count
            },
            "tier_4": {
                "completed": escalation.tier_4_completed is not None,
                "donors_notified": escalation.tier_4_donor_count
            },
            "stock_check": {
                "completed": escalation.blood_bank_checked is not None,
                "blood_bank_units": escalation.blood_bank_units,
                "hospital_stock": escalation.hospital_stock_details
            },
            "total_donors_alerted": escalation.total_donors_alerted,
            "completed_at": escalation.completed_at
        })
    except BloodRequestEscalation.DoesNotExist:
        return Response({"error": "Escalation not found"}, status=404)

```

```

@api_view(["GET"])
@permission_classes([AllowAny])
def admin_notification_logs(request, request_id):
    """Get all notifications sent for a request"""
    logs = NotificationLog.objects.filter(blood_request_id=request_id).order_by('-se

```

```
data = []
for log in logs:
    data.append({
        "id": log.id,
        "donor": log.donor.email if log.donor else "System",
        "tier": log.tier,
        "distance_km": log.distance_km,
        "type": log.notification_type,
        "status": log.status,
        "sent_at": log.sent_at,
        "error": log.error_message
    })

return Response(data)
```

📍 LOCATION 10: Update send_donor_alert() - ADD TIER INFO

CURRENT: blood_requests/views.py, around line 860:

python

```
def send_donor_alert(donor, blood_request, distance):
    subject = "Urgent Blood Donation Needed Near You"
    # ... existing code ...
```

MODIFY TO:

python

```

def send_donor_alert(donor, blood_request, distance, tier=None):
    """Send SMS + Email with optional tier info"""

    sms_message = (
        f"💧 {blood_request.blood_type} blood needed {round(distance, 1)}km away. "
        f"Please login & donate. - RedDrop"
    )

    if donor.phone_number:
        send_sms(donor.phone_number, sms_message)

    tier_text = f" (Tier {tier['tier']})" if tier else ""
    subject = f"💧 URGENT: {blood_request.blood_type} blood needed near you{tier_text}"

    message = f"""Hi {donor.first_name},

{blood_request.blood_type} blood is urgently needed near you.

Hospital      : {blood_request.hospital_location.name if blood_request.hospital_location else ''}
District      : {blood_request.district}
Distance      : {round(distance, 1)} km
Contact       : {blood_request.contact_phone}
Required By   : {blood_request.required_date.strftime('%Y-%m-%d')} if blood_request.required_date

Please login to donate:
http://localhost:5500/donor_dashboard.html

Your help saves lives ❤️
- RedDrop Team"""

    send_mail(
        subject,
        message,
        settings.EMAIL_HOST_USER,
        [donor.email],
        fail_silently=True
    )

```

LOCATION 11: Optional - UPDATE hospital_create_blood_request()

In hospital/views.py (around line 400+):

Add import at top:

```
python  
  
from celery_tasks import orchestrate_tiered_notification
```

After creating the blood request, add:

```
python  
  
# Auto-approve and trigger notifications for hospital requests  
new_request.status = "approved"  
new_request.save()  
  
# Start tiered notifications immediately  
orchestrate_tiered_notification.delay(new_request.id)
```

TESTING CHECKLIST

After all changes:

```
bash
```

```

# 1. Create migrations
python manage.py makemigrations
python manage.py migrate

# 2. Start Redis (in terminal 1)
redis-server

# 3. Start Celery worker (in terminal 2)
celery -A your_project worker --loglevel=info

# 4. Start Django (in terminal 3)
python manage.py runserver

# 5. Test in Django shell (in terminal 4)
python manage.py shell

# Inside shell:
from blood_requests.models import BloodRequest
from adminpanel.views import admin_approve_blood_request

req = BloodRequest.objects.filter(status='pending').first()
response = admin_approve_blood_request(None, req.id)
print(response.data)

# Check if escalation was created
from adminpanel.models import BloodRequestEscalation
esc = BloodRequestEscalation.objects.get(blood_request=req)
print(f"Tier 1 donors notified: {esc.tier_1_donor_count}")
print(f"Total donors notified: {esc.total_donors_alerted}")

```

SUMMARY OF CHANGES

File	Change	Type
adminpanel/views.py	Replace admin_approve_blood_request()	MODIFY
adminpanel/models.py	Add NotificationLog & BloodRequestEscalation	ADD
blood_requests/views.py	Add get_compatible_donors() & update send_donor_alert()	ADD
settings.py	Add Celery & Blood Bank config	ADD

File	Change	Type
your_project/ init.py	Import Celery app	MODIFY
your_project/celery.py	Create Celery config	CREATE
your_project/celery_tasks.py	Create all 6 Celery tasks	CREATE
urls.py	Add escalation status endpoints (optional)	ADD

CRITICAL: Don't Forget!

Install dependencies:

```
bash
pip install celery redis django-celery-beat django-celery-results
```

Start Redis:

```
bash
redis-server
```

Run migrations:

```
bash
python manage.py makemigrations
python manage.py migrate
```

Start Celery worker:

```
bash
celery -A your_project worker --loglevel=info
```

That's it! Your tiered notification system is integrated. 🎉